

SMS Parser Module

Scope

The SMS parsing component was developed to convert the raw `modified_sms_v2.xml` dataset into structured transaction records for use by the API and DSA components.

The implementation was completed in `dsa/parse_xml.py`, approximately 330 lines of Python code, with a corresponding test suite in `dsa/test_parse_xml.py`. The parser was also explained and shared with the other two group members so that the structure, functionality, and usage of the module could be understood and integrated with the other components of the project.

Challenge

The `modified_sms_v2.xml` dataset contains **1,691 SMS records**, but the transaction information is not provided as structured fields. Instead, details such as the transaction type, amount, sender, receiver, and timestamp are embedded within the free-form `body` attribute of each SMS.

An initial audit of the complete dataset identified **13 distinct message templates**. These included English and Kinyarwanda messages as well as incoming and outgoing transactions, failed transactions, reversals, bundle purchases, airtime purchases, bank transfers, agent withdrawals, and OTP messages.

The messages also contained several formatting variations. Some amounts included commas, some messages contained optional phrases such as "`with token`", some included additional phone identifiers, and some did not contain a timestamp in the message body. These differences made it necessary to design specific extraction rules for each template.

Development Approach

The parser was developed incrementally, beginning with an initial audit of the dataset to understand the different message structures.

A temporary classification script was used to examine the SMS bodies and group them into rough template categories. This established the expected patterns and provided a basis for validating the parser.

The extraction logic was then implemented one template at a time. After each template was added, the parser was tested against the complete dataset to check whether the expected records were identified and whether existing classifications remained correct.

Once the extraction logic was working across the dataset, it was organized into `dsa/parse_xml.py` with reusable functions such as `parse_sms_xml()`, `load_transactions()`, `save_json()`, and `is_otp()`.

The completed implementation and its design were then shared with the other group members so that the parser could be used consistently by the API and DSA components.

Key Technical Decisions

Several decisions were made to improve the reliability and usability of the parser.

Timestamp handling:

Human-readable timestamps contained in the SMS body are used whenever available. For messages without a timestamp in the body, the XML epoch timestamp is used as a fallback. Epoch timestamps are converted to `Africa/Kigali` time to remain consistent with the timestamps displayed in the SMS messages.

Retaining failed and reversed records:

Failed transactions, reversals, and the statement-line record are retained rather than discarded. Their state is represented through the `status` field, allowing other components to filter the records when necessary.

OTP filtering:

OTP messages are identified before transaction extraction. Since these messages do not contain the transaction fields required by the parser, they are handled separately rather than being processed through the transaction templates.

Transaction direction:

The value `"self"` is used to represent the user's own account. For outgoing transactions, the sender is `"self"`, while for incoming transactions, the receiver is `"self"`. This provides a simple way to determine transaction direction in downstream processing.

Template matching:

Each template uses multiple identifying markers before extraction is performed. More specific templates are evaluated before generic templates to prevent similar message formats from being incorrectly classified.

For example, airtime messages contain wording similar to ordinary payment messages, so the additional `"to Airtime"` marker is used to distinguish them.

Contiguous identifiers:

Parsed transactions receive sequential IDs beginning at 1. Skipped records do not consume IDs, making the resulting records easier to reference from the API.

Findings During Development

The development process revealed several characteristics of the dataset that affected the final implementation.

The initial rough classifier identified **700 messages as payments**. After stricter extraction rules were applied, the final parser identified **698 completed payments**. The difference was caused by two messages that initially resembled payments but were actually failed bundle purchases. This demonstrated the need for more specific template conditions.

The XML root also declares `count="1693"`, while only **1,691 `<sms>` elements** are present in the file. The parser uses the actual records contained in the dataset and documents the discrepancy rather than creating artificial records.

The initial planning identified approximately six major message patterns. A complete audit revealed **13 templates**, including additional cases such as agent withdrawals, reversals, failed transactions, Kinyarwanda bundle purchases, and a statement-line record.

Bugs Identified and Resolved

Bug	Effect	Resolution
Timezone mismatch	Fallback timestamps were two hours different from timestamps in the SMS body.	Changed timestamp conversion to <code>ZoneInfo("Africa/Kigali")</code> instead of UTC.
<code>bank_transfer</code> pattern too restrictive	Six bank-transfer records could not be extracted.	Updated the regular expression to account for the amount appearing between <code>"transferred"</code> and <code>"to"</code> .
Failed-transaction receiver over-capture	The receiver was extracted as <code>"Bundlesand Packs with token"</code> instead of <code>"Bundlesand Packs"</code> .	Added an optional non-capturing group for <code>"with token"</code> .
Early <code>return</code> inside parsing loop	Processing stopped after the first record.	Moved the <code>return</code> statement outside the <code>for</code> loop.

Testing against the complete dataset was important in identifying these issues because some problems only became visible when the parser was applied to all records.

Results

The completed parser successfully accounted for all **1,691 SMS records** present in the XML:

Parsed transactions: 1,683

Skipped OTP messages: 8

Skipped unrecognized: 0

The parsed transactions were distributed across 13 transaction types:

Transaction Type	Records
payment	698
transfer	585
deposit	248
received	63
direct_payment	36
bundle_purchase	21
airtime	15
bank_transfer	6

failed_transaction	5
--------------------	---

agent_withdrawal	3
------------------	---

reversal_initiated	1
--------------------	---

reversal_confirmed	1
--------------------	---

statement_line	1
----------------	---

Total	1,683
--------------	--------------

The status distribution was:

Status	Records
ok	1,675
failed	5
pending_reversal	1
reversed	1

statement 1

Total 1,683

The complete accounting is:

1,683 parsed + 8 OTP skipped + 0 unrecognized = 1,691 records.

This confirms that every SMS element present in the file was either successfully parsed or explicitly identified as an OTP message.

Testing and Validation

A **35-test suite** was created to validate the parser at both function and dataset levels.

The tests cover:

- OTP identification
- Amount conversion and handling of formatted values
- Epoch timestamp conversion
- Transaction status mapping
- Individual SMS template extraction
- Timestamp fallbacks
- Total parsed record count
- ID continuity
- Required transaction fields
- End-to-end parser integrity

The complete test suite passed successfully:

35/35 tests passed.

Integration and Knowledge Sharing

The completed parser provides the structured data layer required by the other components of the project:

modified_sms_v2.xml → **SMS Parser** → **Structured Transaction Records** → **API / DSA Modules**

The implementation, design decisions, and usage of the parser were explained and shared with the other two group members. This allowed the team to understand how the structured transaction data was produced and how it could be accessed through

`load_transactions()` when integrating the parser with the API and DSA components.

This also helped maintain a consistent data structure across the different parts of the project rather than having individual components interpret the raw SMS dataset independently.